

```
sudo apt-get install sun-java6-jdk sun-java6-plugin
```

Making Sun Java the default

In a terminal, run `sudo update-alternatives—config java` and you should get something like what's shown below:

```
Selection Path
Priority Status
-----
* 0 /usr/lib/jvm/java-6-openjdk/jre/bin/java 1061 auto mode
1 /usr/lib/jvm/java-6-openjdk/jre/bin/java 1061 manual mode
2 /usr/lib/jvm/java-6-sun/jre/bin/java 63 manual mode
Press enter to keep the current choice[*], or type selection
number: 2
```

Select (2) and press `Enter` to choose Sun Java. In the terminal, run `java -version`, which should return the following code:

```
Java version "1.6.0_26"
Java(TM) SE Runtime Environment (build 1.6.0_26-b03)
Java HotSpot(TM) Server VM (build 20.1-b02, mixed mode)
```

Installing required packages

Now run these commands in the terminal:

```
sudo apt-get update
sudo apt-get install git-core gnupg flex bison gperf build- \
essential zip curl \
zlib1g-dev libc6-dev lib32ncurses5-dev ia32-libs \
x11proto-core-dev libx11-dev lib32z1-dev libgl1-mesa-dev g++- \
multilib mingw32 tofrodos python-markdown libxml2-utils
```

Installing and configuring a repo client

Repo is a tool that makes it easier to work with Git in the context of Android. You need a `bin/` directory in your home directory, which is included in your path. Run the following at the terminal:

```
mkdir ~/bin
PATH=~:/bin:$PATH
```

Download the repo script and ensure it is executable:

```
curl https://dl-ssl.google.com/dl/googlesource/git-repo/repo > ~/ \
bin/repo
chmod a+x ~/bin/repo
```

Downloading source code

For the purpose of this article, we will download the source code for Android ICS (Ice Cream Sandwich a.k.a. version 4.0). Type the following in the terminal:

```
mkdir android-x86
cd android-x86
repo init -u http://git.android-x86.org/manifest -b ics-x86
repo sync
```

I recommend that you have at least 10 GB of disk space free, as the source code will take up a lot of space. In case you shut down your computer, or stop the above process, next time just run `repo sync`. The time taken will depend on the speed of your Internet connection. For me, it took almost two whole days. Now, in case you want to download the source code for any other Android branch, just replace `ics-x86` with the appropriate branch name, like `honeycomb-x86`. I would like to point out that for this article I have used the `android-x86` project. You can check out the website to download the source code for different Android branches. Also, you can use the Git mirror server on sourceforge.net to download the source code by using the following command:

```
repo init -u git://android-x86.git.sf.net/gitroot/android-x86/x86/ \
platform/manifest.git -b ics-x86
```

Building the image

This is the most exciting part of the whole process. You need to choose a target for the x86 device you want to use/test. For example, the ICS branch offers the following targets:

- `generic_x86`: for generic x86 PCs/notebooks
- `amd_brazos`: for the AMD Brazos platform
- `eeepc`: for the ASUS EeePC family only
- `asus_laptop`: for some ASUS laptops
- `tegrv2`: for Tegatech Tegrav2 (may work with other Atom N45x-based tablets)

From the above list, select an appropriate target, and use it in this command:

```
make -j4 iso_img TARGET_PRODUCT=asus_laptop
```

Since I have a machine with four cores, I can take advantage of this and speed up my build process by using all the cores. Alternatively, if you don't have a multi-core machine, type the following command:

```
make iso_img TARGET_PRODUCT=asus_laptop
```

This process will take a very long time, depending on your processor. For me, it took over three hours! The generated image is located at `out/target/product/$TARGET_PRODUCT/$TARGET_PRODUCT.iso`. In my case, it was `/home/jayneil/sc/androidx86/out/target/product/laptop/asus_laptop.iso`.

Now you can run the created image using VirtualBox, which can be downloaded from <https://www.virtualbox.org/wiki/Downloads>. Also refer to Figures 2, 3 and 4 for more details.